

ESI2026 - Car Pooling System

Assignment 3

Renan Barbosa de Moraes, Tomáš Magula,
Paulius Sabanskis, Simon Schwitz

April 19, 2026

3 Complete System Architecture

This chapter outlines the complete microservices architecture of the carpooling platform as initially designed, detailing each service's responsibilities and dependencies.

3.1 Service Responsibilities and Relationships

The system is composed of eight independent microservices.

In addition to these business services, the system includes an **API Gateway** and a **Messaging infrastructure** as shared integration components for the whole application. The former provides a unified client entry point and routes requests to all backend microservices, but it does not replace domain-specific services or contain business logic.

1. Authentication Service

Responsibilities: Handles user registration, login, JWT session generation, and role validation (Driver vs. Passenger).

Relationships: Acts as a foundational service. Synchronously queried by the Ride Booking Service and others to validate active user sessions.

2. Profile Service

Responsibilities: Manages general user details, driver verification statuses, and registered vehicle information.

Relationships: Synchronously queried by the Ride Booking Service to verify vehicle ownership. Asynchronously consumes events from the Validation Service to update driver statuses.

3. Ride Booking Service

Responsibilities: The core transactional engine. Manages driver-created rides, passenger seat reservations, and the overall ride lifecycle (pending, completed, etc.).

Relationships: Highly integrated. Synchronously calls Auth, Profile, Geolocation, and Payment services. Asynchronously publishes events when rides are completed. Asynchronously receives events from Payment service.

4. Ride Discovery Service

Responsibilities: Acts as the search engine for passengers, finding, filtering, and ranking available rides based on time and geographical proximity.

Relationships: Acts as an orchestrator. Synchronously aggregates data from the Geolocation, Ride Booking, and Review services.

5. Geolocation Service

Responsibilities: Provides geocoding: converts string addresses to map coordinates, and also coordinates to addresses.

Relationships: Synchronously queried by the Ride Discovery and Ride Booking services.

6. Validation Service

Responsibilities: securely processes and verifies sensitive user documents (driver's licenses, vehicle registration).

Relationships: Operates entirely asynchronously. Publishes validation success/failure events to a message broker for the Profile Service to consume.

7. Payment Service

Responsibilities: Manages credit card authorizations, holds, and final captures for ride payments via external gateways (e.g., Stripe).

Relationships: Synchronously called by Ride Booking to authorize funds. Asynchronously processes final charges and publishes success events.

8. Review Service

Responsibilities: Manages the community feedback loop, allowing users to rate and review drivers after successful rides.

Relationships: Synchronously queried by Ride Discovery to enrich search results with driver ratings.

3.2 Architecture Diagram

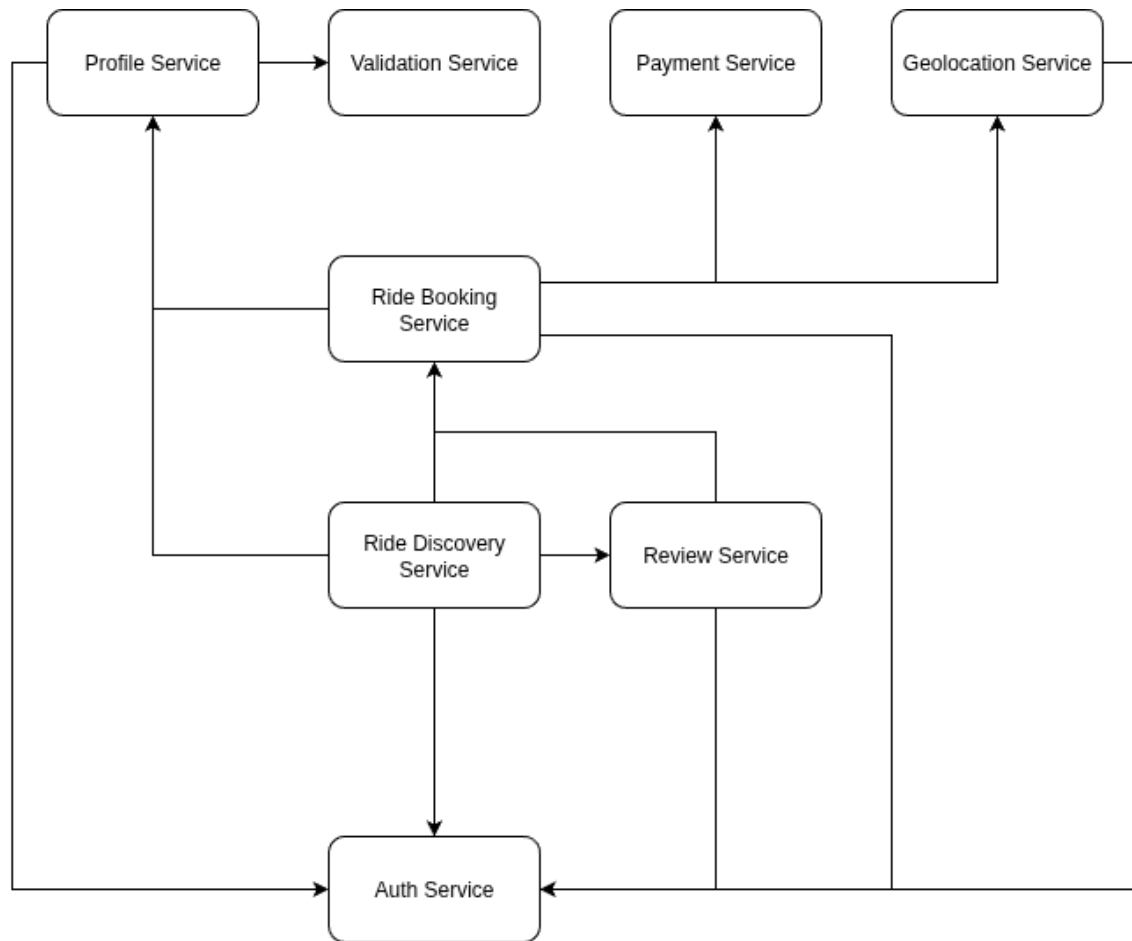


Figure 3.1: Simple Architecture Diagram

3.3 Implementation Scope

To ensure the final system remains realistic and feasible within the project timeline, the team has defined the following set of components to be physically implemented during the development phase.

Services to be Implemented:

- **Simon Schwitz:** Authentication Service, Profile Service
- **Paulius Sabanskis:** Ride Booking Service, Review Service

- **Tomáš Magula:** Ride Discovery Service, Payment Service, API Gateway (whole-application integration component)
- **Renan Barbosa de Moraes:** Validation Service, Geolocation Service, Messaging Infrastructure (whole-application integration component)

All 8 core business services designed in Assignment 1 will be implemented to preserve the platform's complete end-to-end business logic. In addition, an API Gateway will be implemented as a shared integration component to expose a unified entry point while keeping service boundaries intact.

4 Service APIs

4.1 Authentication Service

This service handles secure identity validation and role management.

Table 4.1: Authentication Service REST API Endpoints

Endpoint	Method	Description
/auth/register	POST	Register a new user's credentials and create an initial account.
/auth/login	POST	Authenticate a user and issue a session token (e.g., JWT).
/auth/logout	POST	Invalidate the current user session.
/auth/validate	GET	Validate a user's token (synchronously called by other services).
/auth/roles/{userId}	GET	Retrieve authorization roles (Passenger, Driver) for a user.
/auth/roles/{userId}	PUT	Update user permissions (e.g., upgrading a passenger to a driver role).

4.2 Profile Service

This service manages all personal information, driver states, and vehicle records.

Table 4.2: Profile Service REST API Endpoints

Endpoint	Method	Description
/profiles	POST	Create a new user profile (triggered after successful auth registration).
/profiles/{userId}	GET	Retrieve user profile data.
/profiles/{userId}	PUT	Update basic profile information (name, phone number).
/profiles/{userId}/vehicles	GET	Retrieve all verified vehicles associated with a driver.
/profiles/{userId}/vehicles	POST	Add a new vehicle to the driver's profile.
/profiles/{userId}/status	PUT	Update a driver's verification status (e.g., from pending to verified).

4.3 Ride Booking Service

This service is responsible for creating and managing rides and their reservations.

Table 4.3: Ride Booking Service REST API Endpoints

Endpoint	Method	Description
/rides	POST	Create a new ride.
/rides	GET	Get all rides.
/rides/{rideId}	GET	Retrieve details of a specific ride by its ID.
/rides/{rideId}	PUT	Update ride information.
/rides/{rideId}	DELETE	Delete a ride and cancel it's bookings.
/rides/{rideId}/bookings	POST	Create a new booking for a ride.
/bookings	GET	Get all bookings.
/bookings/{bookingId}	GET	Get booking by id.
/bookings/{bookingId}	DELETE	Delete a booking.

4.4 Ride Discovery Service

This service aggregates and ranks available rides based on passenger search criteria. It acts as a stateless query layer that reads from multiple services without persisting primary data.

Table 4.4: Ride Discovery Service REST API Endpoints

Endpoint	Method	Description
/search	GET	Search rides with criteria (query params: originAddress, destinationAddress, originLat, originLon, destinationLat, destinationLon, departureDate, seatsNeeded, maxPricePerSeat). Either address pair or coordinate pair must be provided.

4.5 Geolocation Service

The Geolocation Service is responsible for transforming human-readable addresses into geographic coordinates.

Table 4.5: Geolocation Service REST API Endpoints

Endpoint	Method	Description
/geocode	GET	Convert address to coordinates
/reverse-geocode	GET	Convert coordinates to address

4.6 Validation Service

The Validation Service ensures that drivers meet regulatory and platform safety requirements by verifying sensitive documents asynchronously.

Table 4.6: Validation Service REST API Endpoints

Endpoint	Method	Description
/validation	POST	Upload documents for validation and start process.
/validation/{requestId}/retry	POST	Retry failed validation.
/validation/{requestId}	GET	Get validation information by id.
/validation/{userId}	GET	Get all validations for a user.
/validation/{vehicleId}	GET	Get all validations for a vehicle.

4.7 Payment Service

This service handles the complete payment lifecycle after a ride is completed, including refund management.

Table 4.7: Payment Service REST API Endpoints

Endpoint	Method	Description
/payments	POST	Initiate a payment for a completed ride.
/payments/{paymentId}	GET	Get payment details.
/payments/{paymentId}/refunds	POST	Request a refund (only on COMPLETED payments).
/payments/{paymentId}/refunds	GET	Get refund details.
/payments/authorize	POST	Authorizes a transaction and places a temporary hold.

4.8 Review Service

This service manages reviews submitted for completed rides.

Table 4.8: Review Service REST API Endpoints

Endpoint	Method	Description
/reviews	POST	Create a new review for a completed ride.
/reviews	GET	Get all reviews.
/reviews/{reviewId}	GET	Retrieve details of a specific review by its ID.
/reviews/{reviewId}	PUT	Update an existing review (e.g., rating, comments).
/reviews/{reviewId}	DELETE	Soft-delete or hide a review.

4.9 API Gateway

This integration component provides a single external entry point for the whole platform and routes calls to all backend services. It contains no domain logic and no persistence.

Table 4.9: API Gateway Public Route Groups

Endpoint	Method	Description
/api/auth/**	ALL	Routes to Authentication Service endpoints.
/api/profiles/**	ALL	Routes to Profile Service endpoints.
/api/rides/**	ALL	Routes to Ride Booking ride endpoints.
/api/bookings/**	ALL	Routes to Ride Booking booking endpoints.
/api/search/**	GET	Routes to Ride Discovery search endpoints.
/api/payments/**	ALL	Routes to Payment Service endpoints.
/api/reviews/**	ALL	Routes to Review Service endpoints.
/api/geolocation/**	GET	Routes to Geolocation Service endpoints.
/api/validation/**	ALL	Routes to Validation Service endpoints.

5 Service Data Models

To ensure complete decoupling, each microservice owns its data model and storage strategy. Stateful services use distinct databases, while stateless services do not persist data.

5.1 Authentication Service Database

This database stores strictly security and access data, keeping sensitive information isolated from general user profiles.

Table 5.1: Authentication Service Data Model

Entity	Attributes	Relationships
UserCredentials	userId (PK, UUID) email (String) passwordHash (String) roles (FK) createdAt (DateTime) updatedAt (DateTime)	1:M with UserRole
UserRole	roleId (PK, UUID) userId (FK, UUID) roleName (String)	Belongs to UserCredentials Deleted with UserCredentials (Cascade)

5.1.1 Session Data Model: Stateless Architecture

This system eschews a traditional server-side session data model in favor of a strictly stateless architecture utilizing JSON Web Tokens (JWT). By enforcing a stateless execution context, the Authentication Service eliminates the overhead of a relational database for persistent session tracking.

Instead, authenticated clients are issued a JWT encapsulating their universally unique identifier (UUID) and Role-Based Access Control (RBAC) claims. Subsequent inter-service requests are authorized autonomously via HMAC-SHA256 cryptographic signature verification. This mathematical validation eliminates I/O to centralized databases, ensuring low-latency communication and horizontal scalability. Consequently, session termination is handled entirely on the client by purging the token artifact from local storage.

5.2 Profile Service Database

This database stores general user details, driver statuses, and vehicle records.

Table 5.2: Profile Service Data Model

Entity	Attributes	Relationships
UserProfile	userId (PK, UUID) firstName (String) lastName (String) driverStatus (String)	1:M with Vehicle
Vehicle	vehicleId (PK, UUID) userId (FK, UUID) make (String) model (String) licensePlate (String)	Belongs to UserProfile

5.3 Ride Booking Service Database

This database stores information about rides and bookings.

Table 5.3: Ride Booking Service Data Models

Entity	Attributes	Relationships
Ride	rideId (PK, UUID) driverId (UUID) vehicleId (UUID) availableSeats (Integer) rideStartDate (DateTime) seatPriceAmount (Decimal) seatPriceCurrency (String) status (String) startLocation (FK) endLocation (FK)	1:M with Booking N:1 with RideLocation (startLocationId, endLocationId)
Booking	bookingId (PK, UUID) rideId (FK) passengerId (UUID) paymentId (UUID) status (String)	Belongs to Ride
RideLocation	rideLocationId (PK, UUID) latitude (Double) longitude (Double) displayAddress (String)	

5.4 Ride Discovery Service Data Model

This service is stateless and does not use a database.

Table 5.4: Ride Discovery Service Data Model

Aspect	Definition
Persistence	No persistent entities or tables.
In-memory cache	No cache is used for search results.
Storage ownership	No local data ownership; data is aggregated on demand from external services.

5.5 Geolocation Service Data Model

This service is stateless and does not use a database.

Table 5.5: Geolocation Service Data Model

Aspect	Definition
Persistence	No persistent entities or tables.
In-memory cache	No cache is used for search results.
Storage ownership	No local data ownership; data is queried on demand from external services.

5.6 Validation Service Database

This database stores information about the requested validations for user documents.

Table 5.6: Validation Service Data Model

Entity	Attributes	Relationships
VerificationRequest	requestId (PK, UUID) userId (UUID) vehicleId (UUID) status (String) isApproved (Boolean)	1:M with Document
Document	documentId (PK, UUID) documentType (String)	Belongs to VerificationRequest

5.7 Payment Service Database

This database persists all payment transactions and refund records with full audit trail.

Table 5.7: Payment Service Data Model

Entity	Attributes	Relationships
Payment	paymentId (PK, UUID)	1:1 with Refund
	bookingId (FK, String)	References Booking [BookingService]
	payerId (UUID)	References User [AuthService]
	payeeId (UUID)	References User [AuthService]
	amount (BigDecimal)	(with Money Value Object)
	currency (String)	
	status (Enum: INITIATED, PROCESSING, COMPLETED, REFUNDED, FAILED)	
	createdAt (DateTime)	
	completedAt (DateTime, nullable)	
Refund	refundId (PK, UUID)	Belongs to Payment
	paymentId (FK)	
	reason (String)	
	refundedAmount (BigDecimal)	
	processedAt (DateTime)	

5.8 Review Service Database

This database stores reviews.

Table 5.8: Review Service Data Model

Entity	Attributes	Relationships
Review	reviewId (PK, UUID)	No relationships
	rideId (UUID)	
	reviewerId (UUID)	
	comment (String)	
	stars (Integer)	

5.9 API Gateway Data Model

This integration component does not own business data and does not maintain a database.

Table 5.9: API Gateway Data Model

Aspect	Definition
Persistence	No persistent entities or tables.
Data ownership	No domain data ownership; request routing only.
State management	Stateless request forwarding and response aggregation where required.

6 System Workflows

To demonstrate both synchronous and asynchronous communication within the architecture, the following integration workflows map out the core operations of the carpooling platform. These workflows highlight how the microservices collaborate to enforce business rules, validate user states, and process transactions.

6.1 Ride Creation

This workflow outlines the process of a driver publishing an upcoming ride to the platform.

Interaction Type: Synchronous (REST API)

Participating Services: Ride Booking Service, Authentication Service, Profile Service, Geolocation Service

Execution Flow:

1. A driver attempts to create a new ride by submitting origin, destination, time, and vehicle details.
2. The **Ride Booking Service** receives the request and makes a synchronous call to the **Authentication Service** to verify the active session token and confirm the user holds a valid "Driver" role.
3. It makes a synchronous call to the **Profile Service** to verify the selected vehicle.
4. It makes a synchronous call to the **Geolocation Service** to translate the provided string addresses into precise map coordinates (geocoding).
5. Once all validations pass, the **Ride Booking Service** persists the new ride in its database and makes it available for discovery.

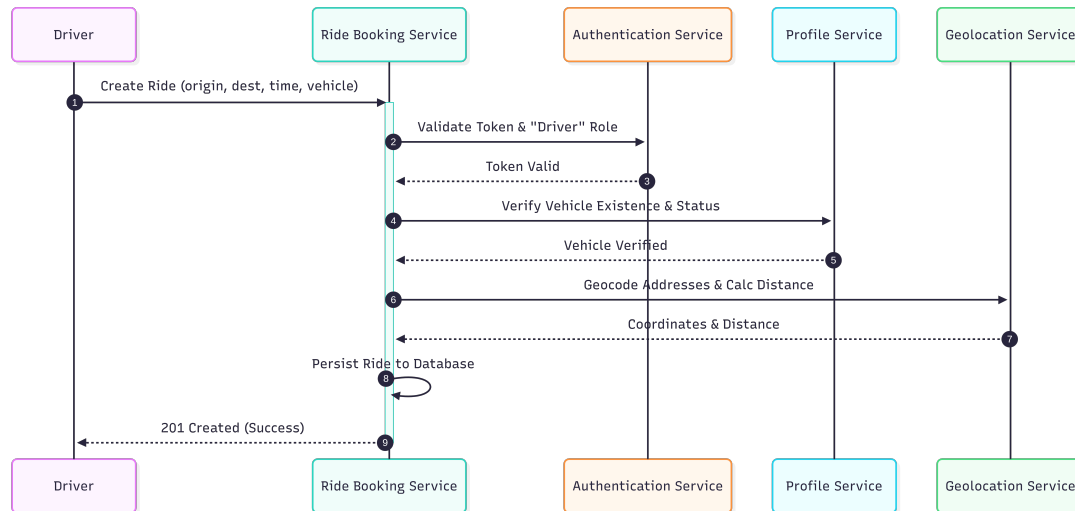


Figure 6.1: Sequence Diagram: Ride Creation Workflow

6.2 Ride Search and Discovery

This workflow describes how a passenger queries the system to find the best available rides.

Interaction Type: Synchronous (REST API)

Participating Services: Ride Discovery Service, Geolocation Service, Ride Booking Service, Review Service

Execution Flow:

1. A passenger submits a search request specifying their desired route and travel time.
2. The **Ride Discovery Service** acts as an aggregator, first calling the **Geolocation Service** to establish a search radius and match geographic proximity.
3. It synchronously queries the **Ride Booking Service** to fetch a list of active rides that are close to the passenger's desired location and currently have unbooked seats.
4. It synchronously queries the **Review Service** to retrieve the community star ratings for the drivers of those specific rides.
5. The **Ride Discovery Service** compiles, filters, and ranks this data before returning the optimized list of rides to the passenger.

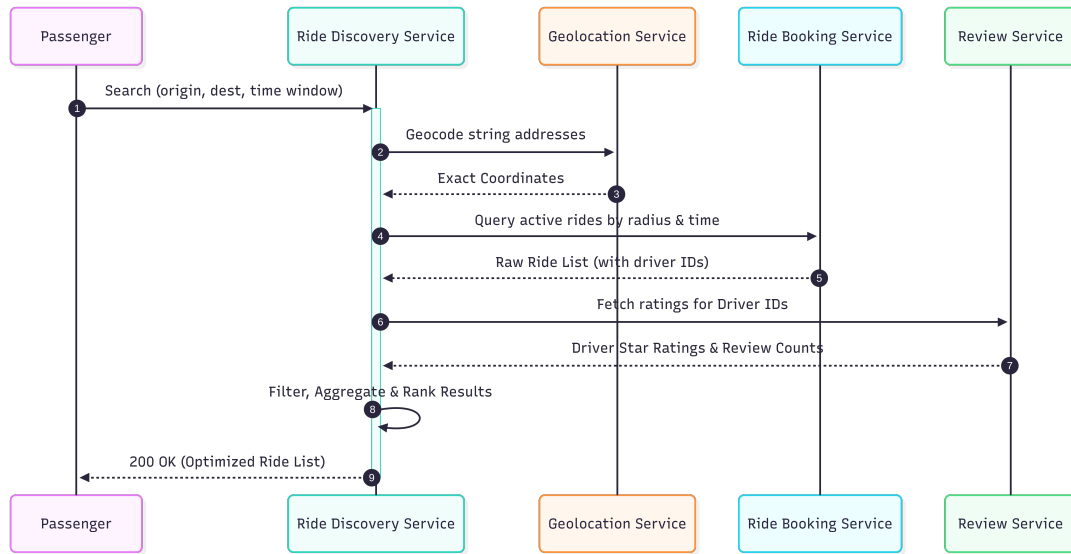


Figure 6.2: Sequence Diagram: Ride Search and Discovery Workflow

6.3 Ride Booking and Reservation

This workflow describes the process of a passenger securing a seat on a selected ride.

Interaction Type: Synchronous (REST API)

Participating Services: Ride Booking Service, Authentication Service, Payment Service

Execution Flow:

1. A passenger selects a specific ride from their search results and initiates the booking process.
2. The **Ride Booking Service** receives the request and synchronously queries the **Authentication Service** to validate the passenger's identity and session.
3. The **Ride Booking Service** checks its own database to ensure the requested seats are still available. If so, it creates a pending "Booking" entity and temporarily locks the seats.
4. It makes a synchronous call to the **Payment Service** to authorize the transaction and place a temporary hold on the passenger's credit card.
5. If the **Payment Service** responds with success, the **Ride Booking Service** confirms the booking and permanently reserves the seats. If the payment fails, the pending booking is dropped, and the seats are unlocked.

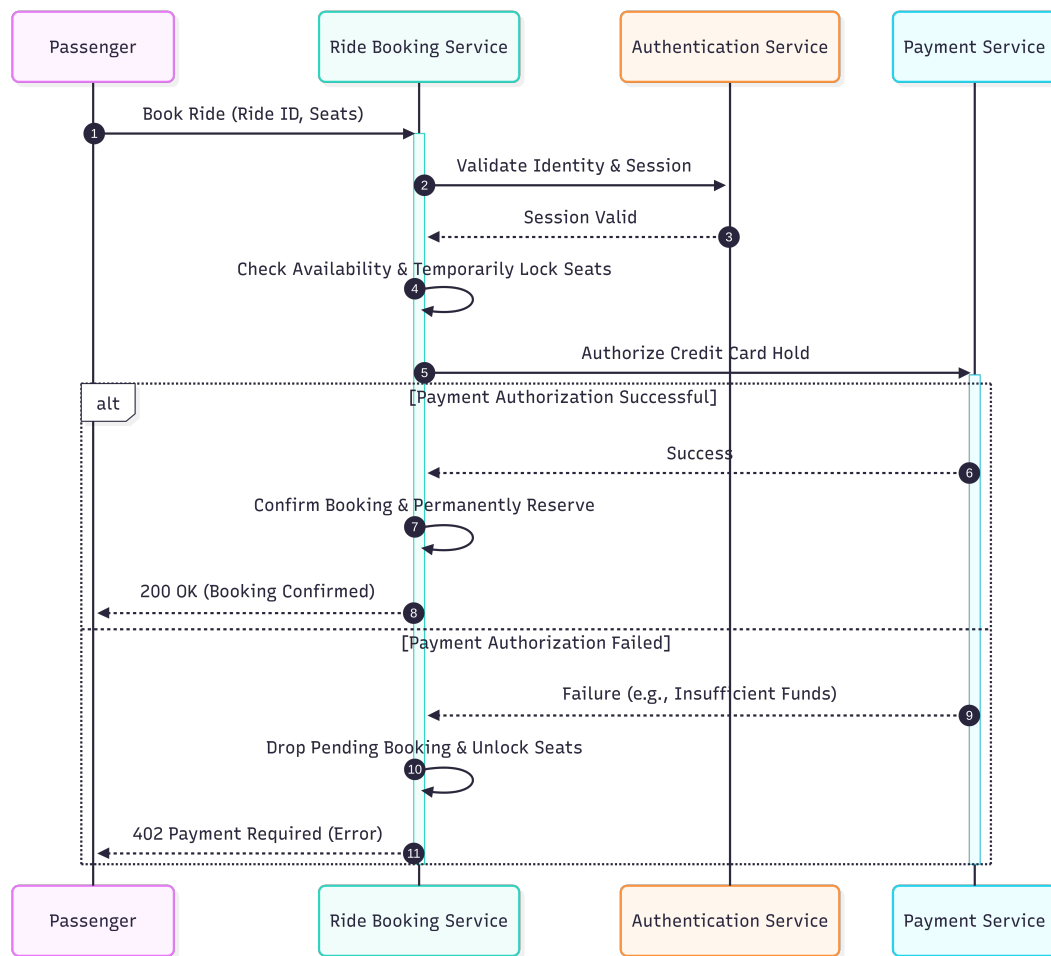


Figure 6.3: Sequence Diagram: Ride Booking Workflow

6.4 Ride Completion and Payment

This workflow handles the post-ride financial transaction and the community feedback loop.

Interaction Type: Asynchronous (Event-driven messaging)

Participating Services: Ride Booking Service, Payment Service, Review Service

Execution Flow:

1. A driver completes the physical drop-off and updates the ride status to "Completed" within the **Ride Booking Service**.

2. The **Ride Booking Service** updates its database and publishes a `RideCompletedEvent` to the message broker.
3. The **Payment Service** consumes this event and executes the final financial charge to the passenger's saved payment method.
4. Once the transaction clears, the **Payment Service** publishes a `PaymentSuccessfulEvent`.
5. The **Ride Booking Service** consumes the payment success event, marking the ride as complete.

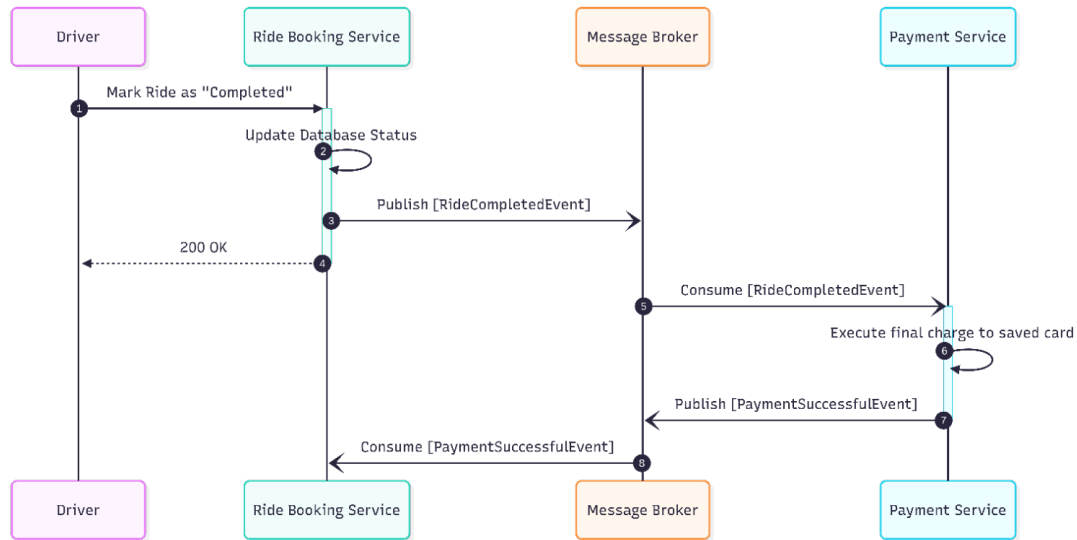


Figure 6.4: Sequence Diagram: Ride Completion and Payment

6.5 Document Validation & Profile Update

This workflow handles the background verification of regulatory driver documents to ensure platform safety.

Interaction Type: Asynchronous (Event-driven messaging)

Participating Services: Validation Service, Profile Service

Execution Flow:

1. A user uploads their driver's license and vehicle registration. The **Validation Service** receives these files and begins processing.
2. Because external document verification is time-consuming, the process runs asynchronously without blocking the user's immediate session.
3. The **Validation Service** concludes its checks and determines an outcome, triggering one of the following asynchronous branches:

- **Validation Success:** The **Validation Service** publishes a `DocumentValidatedEvent` to the message broker. The **Profile Service** consumes this event and updates the user's profile state to "Verified", officially authorizing them to drive.
- **Validation Failure:** The **Validation Service** publishes a `DocumentValidationFailedEvent` to the message broker (including the reason for failure, such as "expired" or "illegible"). The **Profile Service** consumes this event, updates the user's profile state to "Rejected", and flags the account to prompt the user for a re-upload.

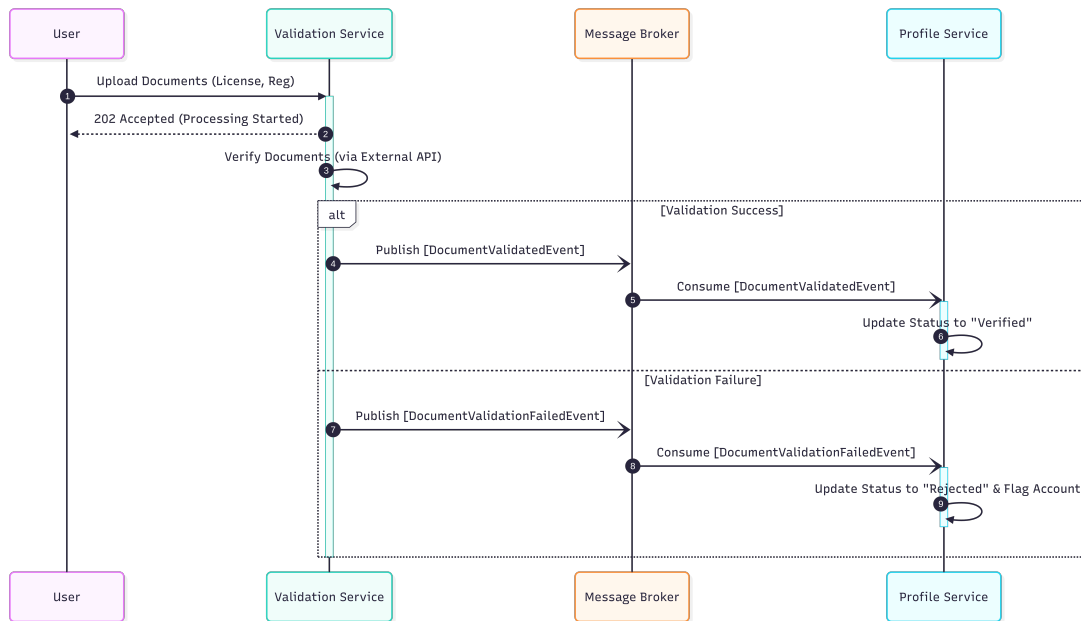


Figure 6.5: Sequence Diagram: Document Validation and Profile Update Workflow

6.6 Review Submission for Completed Ride

This workflow describes the process for a passenger to provide feedback for a specific trip, gated by a confirmed, paid booking.

Interaction Type: Synchronous (REST API)

Participating Services: Review Service, Ride Booking Service, Authentication Service

Execution Flow:

1. **Eligible Booking Retrieval:** The user initiates the process by requesting their history from the **Ride Booking Service**.
2. **History Display:** The **Ride Booking Service** returns the filtered list of eligible bookings to the user interface, ensuring the user can only select a valid, settled trip for review.

3. **Identity Validation:** Upon selecting a booking and submitting the review, the **Review Service** synchronously calls the **Authentication Service** to verify that the submitter's identity and session are valid.
4. **Booking Validation:** To prevent unauthorized submissions, the **Review Service** sends a synchronous request to the **Ride Booking Service** to confirm that the selected booking is indeed complete and belongs to the active user.
5. **Persistence:** After performing internal format validation (e.g., star rating ranges), the **Review Service** persists the feedback in the **Review** table. This data becomes available to the **Ride Discovery Service** for updating driver rankings.

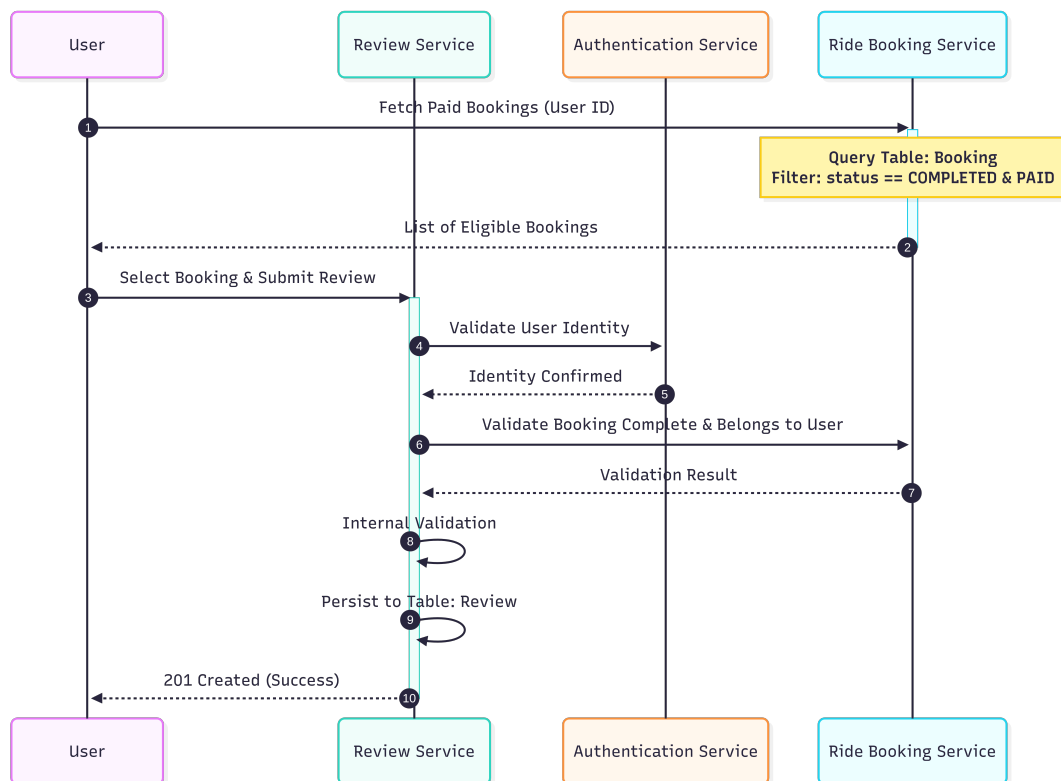


Figure 6.6: Sequence Diagram: Review Submission Workflow

7 Final Implementation Responsibilities

For the implementation phase, the responsibilities are distributed as follows:

7.1 Simon Schwitz

- **Services to Implement:** Authentication Service and Profile Service.
- **Note on Integration Components:** No services were replaced with a shared integration or resilience component for this member. Both assigned services remain strictly business-focused and will manage their own persistent data via separate databases.

7.2 Paulius Sabanskis

- **Services to Implement:** Ride Booking Service and Review Service.
- **Note on Integration Components:** No services were replaced with a shared integration or resilience component for this member. Both assigned services remain strictly business-focused and will manage their own persistent data via separate databases.

7.3 Tomáš Magula

- **Services to Implement:** Ride Discovery Service, Payment Service, and API Gateway.
- **Key Design Decisions:**
 - **Ride Discovery Service:** Stateless aggregation layer with no persistent database and no cache. Computes search results per request by making synchronous HTTP calls to GeolocationService (address geocoding), BookingService (fetch available rides), and ReviewService (fetch driver ratings), then returns ranked results without storing search state.
 - **Payment Service:** Full persistence layer using JPA and PostgreSQL. Manages the complete payment lifecycle from initiation through completion and refunds. Designed to integrate with external payment gateways (Stripe, PayPal, etc.). Consumes RideCompletedEvent asynchronously and publishes PaymentSuccessfulEvent.
 - **API Gateway:** Provides a unified external API for frontend clients and routes requests to all platform services. It centralizes cross-cutting concerns such as header forwarding and request routing while keeping business logic inside the owning services.

7.4 Renan Barbosa de Moraes

- **Services to Implement:** Geolocation Service, Validation Service and Messaging/Event infrastructure.
- **Note on Integration Components:** Since the implementation of the geolocation service is very simple, a shared messaging component will also be implemented. In any case, the geolocation microservice will still exist, with a very simple API and third-party integration.